

Other Things to Include

- Stochastic Gradient Descent; what it is, and compare it to Gradient Descent
- K-Fold Cross Validation; what it is, and how it compares to not using it
- L1 norm regularization (Lasso Regression)
 - Put this next to ridge regression
- Support Vector Machines: what they are by math definition + what they do
 - $w^T x_i + b \geq 1$ if $y_i = +1$ (class 1)
 - $w^T x_i + b \leq -1$ if $y_i = -1$ (class 2)
 - Margin (distance between hyperplanes) is dependent on w and decreases with length of w .
- K-means; important formulas and what it means, and how it's done

Lagrangian Optimization

- $L(x, \alpha, \beta) = f_0(x) + \sum \alpha_i g_i(x) + \sum \beta_j h_j(x)$
 - Each constraint has a set of "Lagrangian" multipliers
 - We want to get the maximum of this function L over the α and β values, with $\alpha_i \geq 0$
- Dual Problem: minimizing over x and then over α and β (all vectors by the way)
 - $g(\alpha, \beta) = \text{minimum over } x \text{ of } L(x, \alpha, \beta)$, then
 - $d^* = \max \text{ over } \alpha, \beta \text{ such that } \alpha_i \geq 0 \text{ of } g(\alpha, \beta)$
 - If x is feasible, we get $d^* \leq p^*$ where p^* solved the primal problem and d^* solves the dual problem.

MSE Loss Function

- $J(\theta) = 1/N$ times the sum of $((\theta^T x_i) - y_i)^2$
- We can rewrite this as $J(\theta) = \text{L2 norm of } (X^T \theta - Y)$
- Ridge regression formula
- Sigmoidal regression formula

Linear Decision Boundary

- $y_i \text{ times } (w^T x_i + b) \geq 1$
- Perceptron Algorithm
 - Update w_k if $w_k^T x_i$ less than or equal to 0; o.w. don't
 - Repeat this until $w_k^T x_i \geq 1$

Linear Classifier

- $h(x_i) = \text{sign}(w^T x + b)$ = predicted label $\hat{y}_i$ by the classifier
- $\hat{y}_i$ is equal to 1 if $w^T x_i + b > 0$, and -1 otherwise
- We can combine the b into the w by appending a 1 to the feature vector

Loss Function

- $\text{argmin}_{\theta} \text{ of } (1/N) * \sum_{i=1}^N (h_{\theta}(x_i) - y_i)^2$
 - This is the squared error loss function

Maximum Likelihood Expectation

- $1/N * (\sum_{i=1}^N \log(y_i * h_w(x_i) + (1-y_i)(1-h_w(x_i))))$
- Then, we want w^* as argmin_{w} over w of the negative of that expression.

Log-Likelihood

- We take $h_w(x) = g(w^T x) = 1/(1 + \exp(-w^T x + b))$
- We maximize the MLE equation over the vector w with this as $h_w(x_i)$
 - Or, we minimize the negative of this function, so we can take the negative of this function to be the loss function.
- To get the w^* optimal weights, we need partial of the loss function with respect to $w = 0$, and the hessian semi-definite for minimization.

Linear Regression

- $\hat{y}_i = w^T x_i$, and now the loss function is the following:
 - $L_w(x, y) = 1/N * \sum (y_i - \hat{y}_i)^2$, which is MSE
 - Again, to get w^* , we take the gradient and set it to 0.
- Solution for linear regression was:
 - $w^* = (X^T X)^{-1} X^T y$
 - This also equals $R_x^{-1} R_{xy}$ where R_{ab} = covariance matrix between vectors a and b

Covariance Matrix

- Include how to calculate the covariance matrix for all the data points
 - This tells you how correlated the data points are with each other.

Minimizing a Function

- Hessian must be positive definite; this is the analog of second derivative > 0
- Include formula for the Hessian matrix

Gradient Descent

- $w_{(k+1)} = w_k - (\text{step size})_k * \text{gradient with respect to } w \text{ of the function } f, \text{ evaluated at } w_k$
 - Repeat this until the stopping criterion is met; for example, if $f(\text{new weights}) - f(\text{old weights})$ is within some epsilon, or very small.

Linear Algebra Rules

- Transpose of $(A \text{ times } B) = (B \text{ transpose times } A \text{ transpose})$
- LP norm equals p-th root of $(x_1^p + x_2^p + \dots)$
- L2 norm of x is equal to $(x \text{ transpose} * x)$
- Reordering (vector) times (matrix) times (vector)
 - $xAy = (y \text{ transpose}) A (x \text{ transpose})$ only if A is symmetric
- Meaning of Positive Semi-Definite, Positive Definite, and “Positive” and “Definite”

Gradients

- How to formulate the Hessian matrix

1. Matrix and Vector Calculus

Since you are working with logistic regression and gradient descent, these identities are critical:

- **Gradient of Linear Terms:** $\nabla_w (w^{\text{top}} x) = x$.
- **Gradient of Quadratic Forms:** $\nabla_w (w^{\text{top}} A w) = (A + A^{\text{top}})w$. If A is symmetric, this simplifies to $2Aw$.
- **Chain Rule for Gradients:** $\nabla_w f(g(w)) = f'(g(w)) \cdot \nabla_w g(w)$.

2. Logistic Regression & Sigmoid Identities

- **Sigmoid Function:** $\sigma(z) = \frac{1}{1 + e^{-z}}$.
- **Sigmoid Symmetry:** $1 - \sigma(z) = \sigma(-z)$.
- **Sigmoid Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Decision Boundary:** For logistic regression, the boundary is $w^{\text{top}} x = 0$.
- **Binary Cross-Entropy (BCE) Loss:** $L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$.

3. Optimization and Gradients

- **Gradient Descent Update:** $w_{t+1} = w_t - \eta \nabla L(w_t)$, where η is the learning rate.
- **Hessian Matrix (H):** A matrix of second-order partial derivatives. If H is Positive Semi-Definite ($\lambda_{\min}(H) \geq 0$), the function is convex.
- **Convexity Requirement:** For a local minimum to be a global minimum, the loss function must be convex.

Vector Differentiation

- We can take a derivative of a scalar with respect to a vector, where the scalar is a dot product of two vectors (for example, $A^T B$).
- Denominator-Size-Rule: if we have $d(\text{something in } \mathbb{R}^1)/d(\text{something in } \mathbb{R}^d)$, then we want the output to be in $\mathbb{R}^{d \times 1}$ (a simple vector of size d).
 - This is how we got to “gradient of $(\theta^T x)$ with respect to $\theta = x$ ”
 - We also got to gradient with respect to x of $(x^T y) = \text{gradient with respect to } x \text{ of } (y^T x) = y$
 - We also got **gradient of $\|x\|_2^2$ with respect to $x = \text{gradient with respect to } x \text{ of } (x^T x) = 2x$.**

Kernels

- Formula for the Gram Matrix (the Kernel)

General Formulation

For a set of n vectors $v_1, \dots, v_n$ in an inner product space, the Gram matrix G is:

$$G = \begin{bmatrix} \langle v_1, v_1 \rangle & \langle v_1, v_2 \rangle & \dots & \langle v_1, v_n \rangle \\ \langle v_2, v_1 \rangle & \langle v_2, v_2 \rangle & \dots & \langle v_2, v_n \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle v_n, v_1 \rangle & \langle v_n, v_2 \rangle & \dots & \langle v_n, v_n \rangle \end{bmatrix}$$

- This gives the value of the kernel function for any two vectors in the space
- Positive Semi-Definiteness
 - If any eigenvalue is negative, then it is not PSD
 - Alternatively, if the determinant is negative, then it is not PSD
 - This is because the determinant of a square matrix is the product of the eigenvalues
- Kernel Composition Rules for Positive-Semi-Definiteness
 - Integer Powers, Polynomial Kernels, Exponentiation
 - Sum Rule, Product Rule, Scaling Rule
- Schur Product Theorem

4. Kernel Methods

- **Mercer's Condition:** A kernel $K(x, x')$ is valid if its Gram Matrix G ($G_{ij} = K(x_i, x_j)$) is Symmetric Positive Semi-Definite.
- **Common Kernels:**
 - **Linear:** $K(x, x') = x^{\top} x'$.
 - **Polynomial:** $K(x, x') = (x^{\top} x' + c)^d$.
 - **Gaussian (RBF):** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$.

3. Method of Construction (Algebraic Rules)

You can determine validity by checking if a function is composed of known valid kernels (K_1 and K_2). The following operations preserve kernel validity:

- **Summation:** $K(x, x') = K_1(x, x') + K_2(x, x')$
- **Product:** $K(x, x') = K_1(x, x') \cdot K_2(x, x')$
- **Scaling:** $K(x, x') = c \cdot K_1(x, x')$ for any constant $c > 0$
- **Feature Mapping:** $K(x, x') = f(x) \cdot K_1(x, x') \cdot f(x')$ for any function $f(x)$